

Final Project: Well-Architected Framework Analysis

Course Number CST110/299 – Introduction to Cloud Computing

Professor Jason Wright

7/21/26

Executive Summary

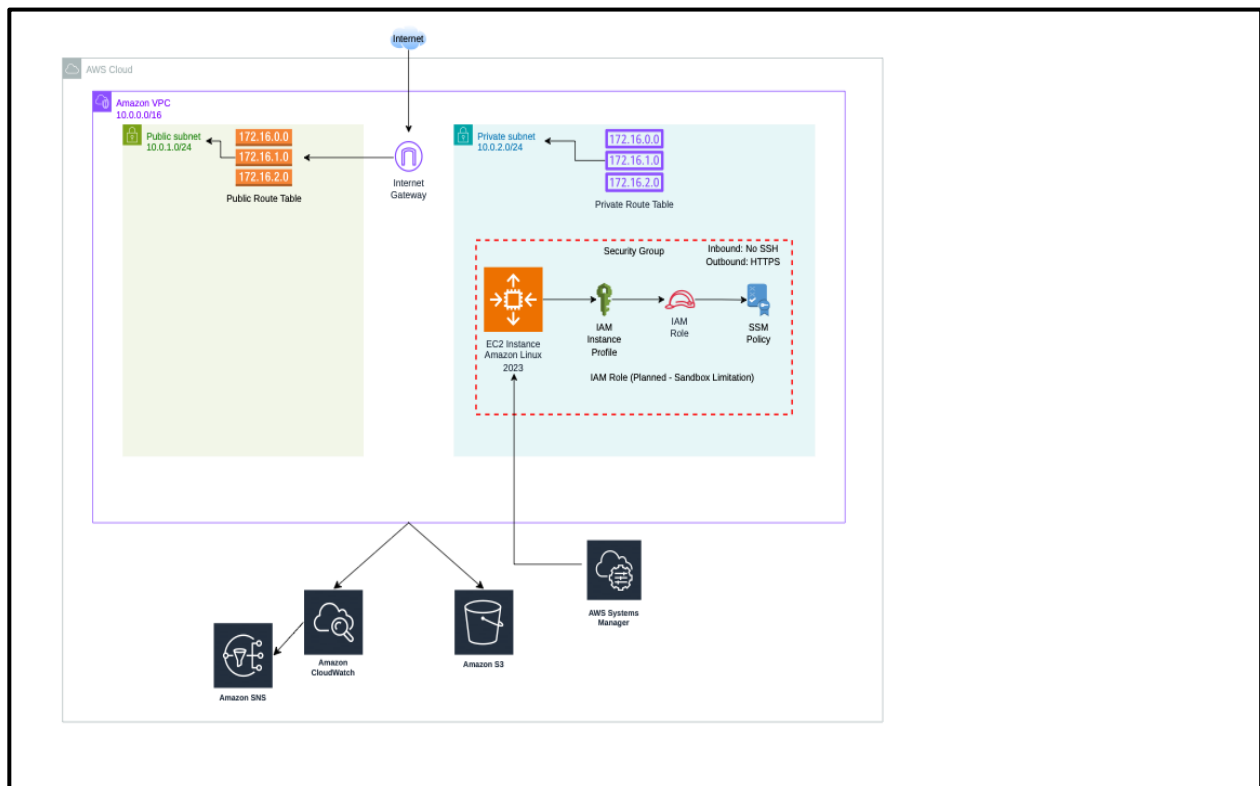
This project demonstrates the design and implementation of a secure, automated, and fully reproducible cloud infrastructure on Amazon Web Services (AWS) using Infrastructure as Code (IaC) principles. The solution address three core business challenges faced by a small organization: the need for consistent and repeatable cloud environments, lack of continuous operational monitoring, and the requirement for controlled and version-managed infrastructure changes.

The environment is provisioned using Terraform to deploy AWS resources, including Virtual Private Cloud (VPC), public and private subnets, routing components, Amazon EC2, Amazon S3, Amazon CloudWatch, and Amazon Simple Notification Service (SNS). Infrastructure is maintained and validated through a GitHub Actions continuous integration (CI) workflow. This ensures development, staging, and production environments can be deployed consistently without manual configuration, reducing human error and supporting repeatable infrastructure deployments.

The solution integrates Infrastructure as Code using Terraform to support automation, version control, and repeatability. Automated monitoring through CloudWatch log groups and metric alarms use SNS automated email notifications. The architecture was designed following AWS security best practices of network segmentation, least-privilege principles, and the planned use of IAM roles. Due to permission restrictions within the AWS sandbox learner environment IAM

resources could not be deployed but were documented as part of the recommended production architecture. This solution is designed to reflect real-world AWS architectural principles, with a strong emphasis on security, operational excellence, and reproducibility.

Network Architecture

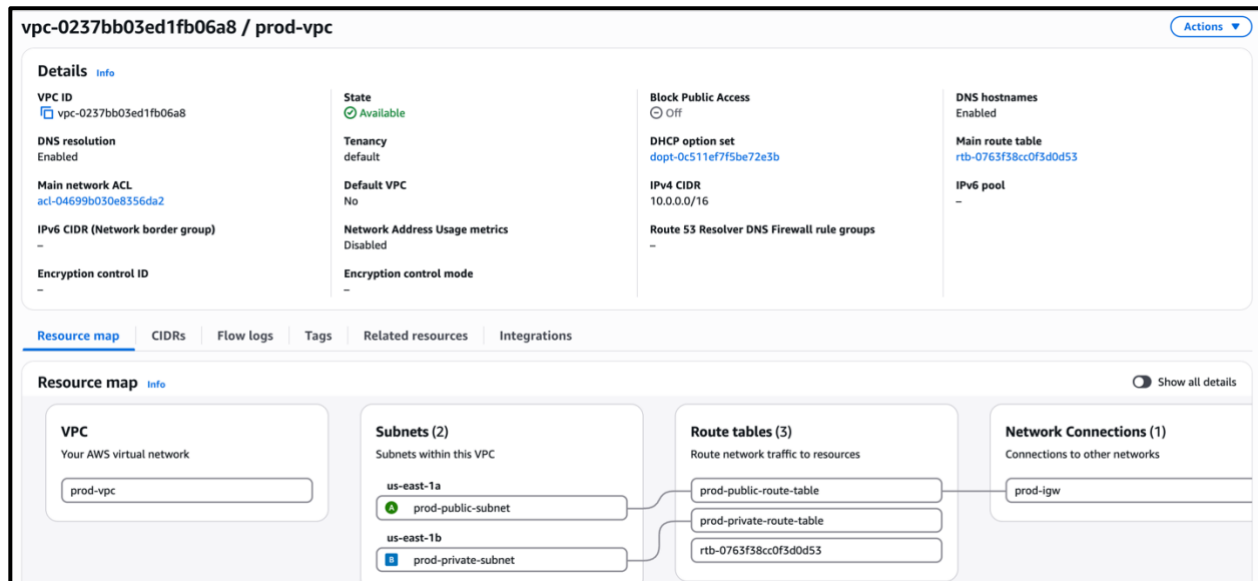


Network Architecture Diagram

Technical Architecture Review

The completed solution implements a modular cloud infrastructure on Amazon Web Services (AWS) using Infrastructure as Code (IaC) with Terraform. The architecture was designed to provide a secure, repeatable, and maintainable environment for a small organization's cloud workloads while following AWS best practices. Resources are organized into reusable Terraform modules for networking, compute, monitoring, and storage which allows the same codebase to support multiple deployments with minimal configuration changes.

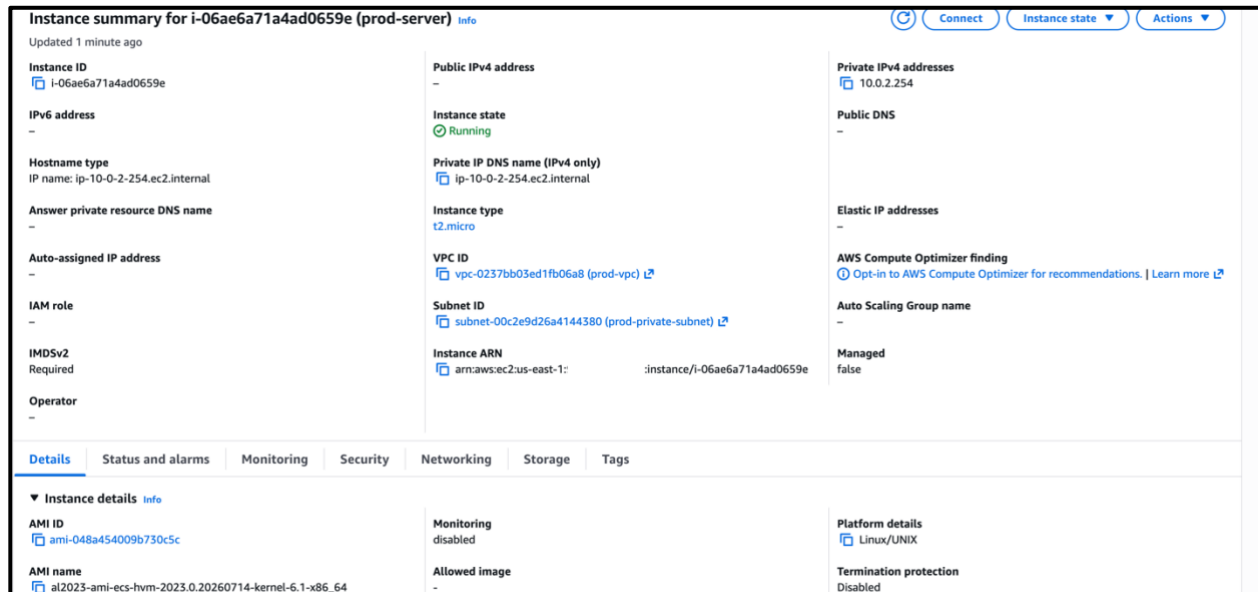
The networking layer establishes a Virtual Private Cloud (VPC) using the private address space 10.0.0.0/16. The VPC contains separate public and private subnets that isolate resources according to their level of accessibility. The public subnet contains networking components that require internet connectivity, including the Internet Gateway and public route table. The private subnet hosts an Amazon EC2 instance and security controls. Separate route tables provide independent routing policies for each subnet. The decision was intentional to follow the principle of network segmentation and to reduce exposing internal resources unnecessarily.



VPC Resources

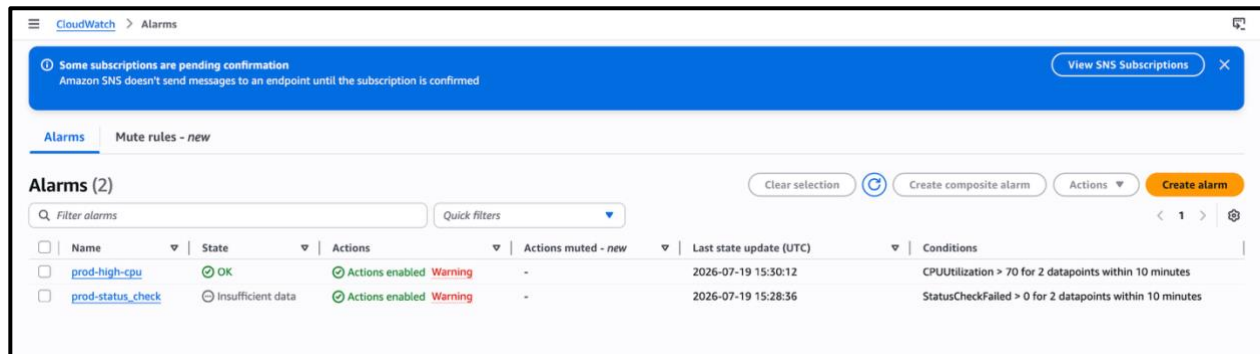
The compute layer consists of an Amazon EC2 instance deployed within the private subnet. Creating the instance outside the public network reduces attack surface by preventing direct internet access. Network access is controlled through a dedicated security group following the principle of least privilege by allowing only the minimum outbound communication required for AWS services. The production architecture includes an IAM role and instance profile to provide temporary AWS credentials and secure System Manager administration without the use of SSH keys. Due to AWS Academy Learner Lab restrictions deploying IAM resources was not possible. These components remain part of the recommended production design and architecture diagram. Due to the time-limited nature of the AWS sandbox and project scope, implementation focused on validating the production environment. The reusable Terraform modules and

environment specific configuration files were intentionally designed to support future expansion without requiring architectural changes.



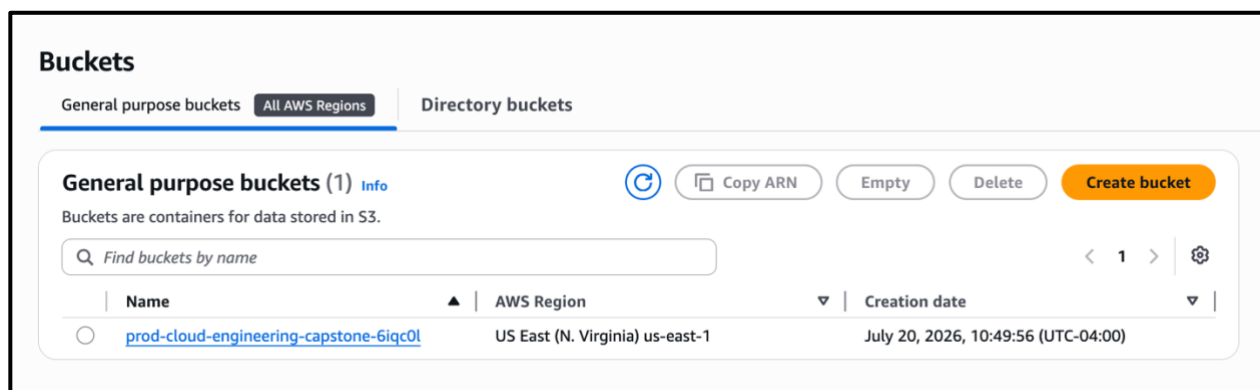
EC2 Instance

Monitoring is implemented through a dedicated Terraform module. CloudWatch provides centralized log storage, collects metrics, and evaluates CloudWatch alarms monitoring CPU utilization and EC2 instance health. Simple Notification Service (SNS) sends automated email notifications when alarm thresholds are exceeded, providing administrators with awareness of operational events.



CloudWatch Alarms

Persistent object storage is provided through Amazon S3. The module provisions a uniquely named S3 bucket using Terraform and enables server-side encryption with AES-256 to protect data at rest. During implementation, the AWS sandbox successfully created the bucket but denied Terraform permission to retrieve Object Lock configuration metadata. This prevented Terraform from completing the resource lifecycle. This limitation was documented as part of the project's implementation constraints stemming from sandbox restrictions rather than Terraform configuration.



S3 Bucket

Infrastructure deployment is fully automated through Terraform, while source code management is maintained in GitHub. A GitHub Actions workflow performs automated formatting, initialization, and validation of Terraform code when changes are committed to the repository. The original project proposal included a fully automated deployment pipeline using AWS native CI/CD services. This was revised due to IAM restrictions within the sandbox environment. GitHub Actions was chosen because it reflects industry practices while allowing automatic infrastructure validation to occur.

A screenshot of a terminal window with a dark background and light text. The window title bar shows 'Problems Output Debug Console Terminal Ports' and a 'bash -prod' icon. The terminal content shows the output of a Terraform apply command. It starts with a warning that only 'yes' will be accepted, followed by a prompt 'Enter a value: yes' and the response 'yes'. The output then lists various AWS resources being created, such as CloudWatch log groups, SNS topics, VPCs, subnets, and EC2 instances, along with their creation times and IDs. At the bottom, a green line indicates 'Apply complete! Resources: 15 added, 0 changed, 0 destroyed.'

Terraform Validation

Several decisions were made to balance realism with the limitations of the sandbox environment. The infrastructure uses a single Availability Zone to reduce resource consumption while maintaining a modular design capable of future project growth. A NAT Gateway was intentionally omitted to simplify the deployment and remain within the constraints, although it

would normally be recommended for a project requiring outbound internet access from a private resource.

The completed architecture demonstrates a secure, modular, and reproducible cloud infrastructure that integrates networking, compute, storage, monitoring, IaC, and CI into a unified solution. While some production features could not be fully implemented due to sandbox restrictions, the final design reflects industry standard cloud engineering practices and provides a strong foundation for future expansion.

Pillar Analysis

The project emphasizes operation excellence through IaC, version control, and automated validation. Terraform provisions all infrastructure from reusable modules ensuring deployments are consistent and repeatable while reducing configuration errors. The repository is organized into separate networking, compute, monitoring, and storage modules allowing infrastructure to be maintained using separation of concerns. GitHub Actions automatically perform Terraform formatting, initialization, and validation when changes are committed. CloudWatch collects metrics and logs. CloudWatch Alarms and SNS work together to provide automated notifications when CPU utilization or instance health thresholds are exceeded.

Security was incorporated throughout the project using a defense-in-depth approach. Network segmentation separates the public networking components from private workloads through

subnets and route tables. Security groups follow least-privilege principles by denying inbound traffic and allowing only the required outbound HTTPS communication. Amazon S3 storage uses server-side AES-256 encryption to protect data at rest. The original and documented architecture included IAM roles and AWS Systems Manager Session Manager to eliminate SSH access and avoid long-lived credentials.

Reliability was designed into the infrastructure through Terraform, allowing environments to be recreated consistently from source code. CloudWatch monitoring and automated SNS notifications improve operational reliability by detecting performance and health issues before they become critical failures. While this project uses a single Availability Zone due to sandbox limitations, a production deployment would distribute resources across multiple Availability Zones and include redundant networking components to improve fault tolerance and availability.

To consider performance efficiency the project provisions infrastructure appropriate for the requirements of a small organization while maintaining flexibility for future growth. The virtual network uses a 10.0.0.0/16 CIDR block to provide sufficient address space for expansion.

Modular Terraform code allows compute, monitoring, and storage resources to scale independently as requirements change. Resource sizing was intentionally chosen to align with project requirements and sandbox limitations. Monitoring through CloudWatch provides visibility into CPU utilization and system health, allowing for data-driven decisions rather than reactive troubleshooting.

Cost optimization influenced several architectural decisions. Resource selection was intentionally minimal, a single EC2 instance, one VPC, one public subnet, one private subnet, and only the services required to demonstrate the solution. Terraform allows resources lifecycle management by creating and destroying environments on demand, preventing unnecessary resource consumption. The design intentionally omitted higher-cost production components such as NAT Gateways and multiple Availability Zone deployments while maintaining modular architecture that can support future expansion.

Sustainability is supported through automation, resource efficiency, and infrastructure reuse. IaC eliminates repetitive manual provisioning while allowing environments to be recreated consistently using the same Terraform modules. Individual components can be updated or reused without redesigning the entire solution. The project minimizes unnecessary cloud resources by deploying only services required to satisfy the business requirements. This reduces infrastructure complexity, limits resource consumption, and demonstrates efficient cloud engineering practices while maintaining a path for future modifications.

Conclusion

This project successfully demonstrated the design and implementation of a secure, automated, and reproducible cloud infrastructure using AWS and IaC principles. The completed solution demonstrates practical application of the AWS Well-Architected Framework. The resulting infrastructure is modular, reproducible, and designed for future expansion. Several implementation challenges were encountered due to sandbox limitations. Rather than redesign

the architecture, alternate solutions were implemented in order to preserve AWS best practices while reflecting the realities of working within restricted enterprise environments.

The complete project repository containing Terraform source code, architecture diagrams, implementation screenshots, and supporting documentation is available at:

<https://github.com/Vukodlok/aws-cloud-engineering-capstone>

References

- Amazon Web Services. (2024). *Amazon CloudWatch User Guide*.
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>
- Amazon Web Services. (2024). *Amazon EC2 User Guide*.
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/>
- Amazon Web Services. (2024). *Amazon S3 User Guide*.
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
- Amazon Web Services. (2024). *Amazon Simple Notification Service Developer Guide*.
<https://docs.aws.amazon.com/sns/latest/dg/>
- Amazon Web Services. (2024). *Amazon VPC User Guide*.
<https://docs.aws.amazon.com/vpc/latest/userguide/>
- Amazon Web Services. (2024). *AWS Well-Architected Framework*.
<https://docs.aws.amazon.com/wellarchitected/latest/framework/>
- GitHub. (2024). *GitHub Actions documentation*. <https://docs.github.com/actions>
- GitHub. (2024). *GitHub Projects documentation*. <https://docs.github.com/issues/planning-and-tracking-with-projects>
- HashiCorp. (2024). *Terraform documentation*. <https://developer.hashicorp.com/terraform/docs>
- HashiCorp. (2024). *Terraform AWS Provider documentation*.
<https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
- CloudTechExec. (2026). *[Cloud Engineers: Build These 3 projects Before You Apply for Any Job]* [Video]. YouTube. <https://www.youtube.com/watch?v=TN830wef2zM>